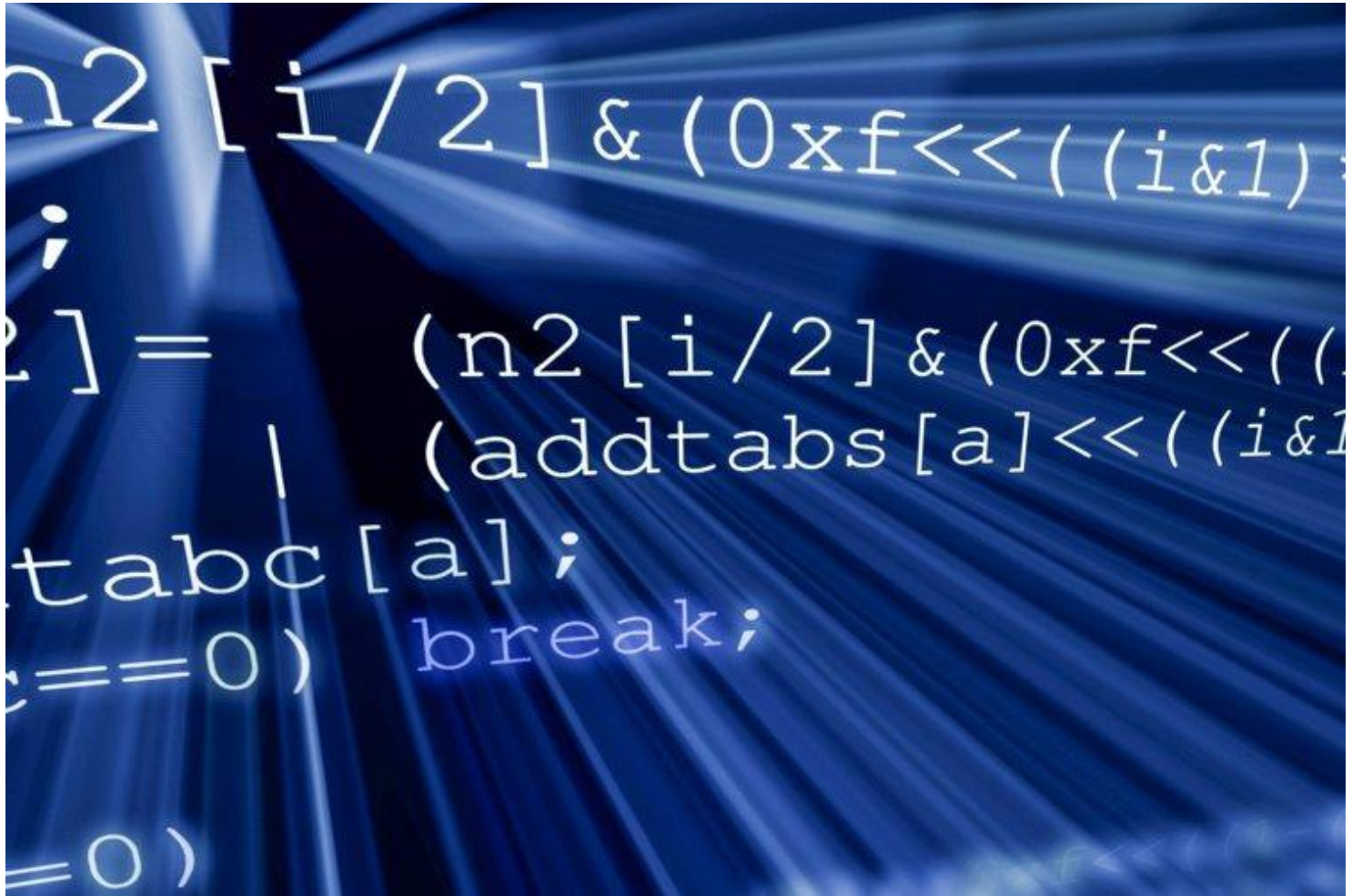




MANUAL TÉCNICO

Generador de Analizadores Sintácticos LL(1) - Lenguaje Wison

1. Arquitectura del Sistema

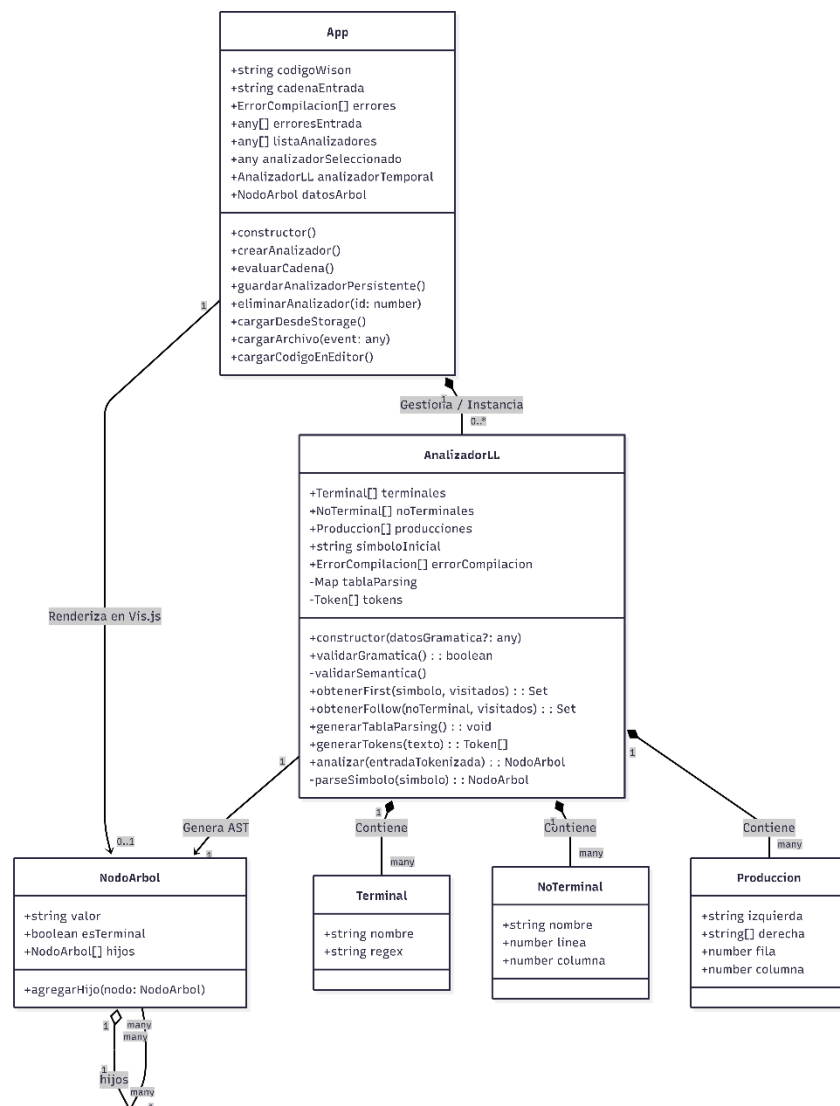
La aplicación web fue desarrollada utilizando una arquitectura basada en componentes del lado del cliente.

- **Frontend:** Desarrollado en Angular con TypeScript, haciendo uso de Bootstrap para el diseño responsivo de la interfaz.
- **Analizador Léxico y Sintáctico Base (Meta-Analizador):** Se empleó la herramienta **Jison** para procesar los archivos de configuración .wison y extraer las estructuras gramaticales.

- **Motor LL(1) Propio:** Se construyó una clase principal en TypeScript (AnalizadorLL) que toma los datos de Jison, genera dinámicamente el autómata léxico (mediante expresiones regulares dinámicas) y construye la tabla de análisis sintáctico descendente.
- **Generación del AST:** Se integró la librería gráfica **Vis.js** para la renderización interactiva del Árbol de Derivación (Abstract Syntax Tree).

2. Diagrama de Clases

- El núcleo del motor compilador se basa en un diseño orientado a objetos para representar los componentes de la gramática.



Descripción de las Clases Principales:

- **App (Controlador Principal):** Es el componente raíz de la aplicación Angular. Actúa como el puente entre la interfaz de usuario (HTML/CSS) y la lógica del compilador. Se encarga de gestionar el estado de la aplicación, manejar los eventos de los botones (como cargar archivos o evaluar cadenas), mantener la persistencia de datos mediante el localStorage y coordinar la renderización de errores y del Árbol de Derivación (AST) generado por el motor.
 - **AnalizadorLL:** Es el motor principal. Encapsula toda la lógica para verificar recursividad, calcular los conjuntos *FIRST* y *FOLLOW*, detectar colisiones en la tabla de parsing y efectuar las llamadas recursivas para consumir los tokens.
 - **NodoArbol:** Estructura de datos recursiva que almacena la información de cada nodo del AST, incluyendo su valor (lexema) y referencias a sus nodos hijos.
-

3. Procedimiento para Construir las Gramáticas

Para que el analizador generado sea capaz de procesar cadenas correctamente, la gramática definida por el usuario en el archivo. Wison debe someterse al siguiente procedimiento teórico y práctico:

1. **Definición Léxica:** Se identifican los componentes léxicos (Terminales) y se les asocia una expresión regular o cadena literal. En Wison, se manejan operaciones unitarias como $*$, $+$ y $?$.
2. **Definición Sintáctica (Eliminación de Ambigüedad):** Dado que se utiliza un método de análisis descendente (LL1), la gramática original debe transformarse para cumplir dos reglas estrictas:
 - **Eliminar la recursividad por la izquierda:** Si una regla tiene la forma $A \rightarrow A\alpha|\beta$, debe reestructurarse usando un nuevo no terminal A' para que quede como $A \rightarrow \beta A'$ y $A' \rightarrow \alpha A'|\epsilon$.
 - **Factorización por la izquierda:** Si dos producciones comparten el mismo prefijo ($A \rightarrow \alpha\beta|\alpha\gamma$), se debe extraer el factor común para evitar que el analizador tenga que "adivinar" qué camino tomar.

3. **Cálculo de FIRST y FOLLOW:** El sistema evalúa qué terminales pueden aparecer al principio de una derivación (*FIRST*) y qué terminales pueden seguir a un no terminal (*FOLLOW*).
 4. **Construcción de la Tabla M:** Se cruzan los No Terminales con los Terminales. Si la gramática fue correctamente factorizada en el paso 2, cada celda de la tabla contendrá como máximo una regla. Si hay más de una, el sistema reportará una "Colisión" y rechazará la gramática.
-

4. Explicación de las Gramáticas Utilizadas

Durante el desarrollo y pruebas del sistema, se utilizaron dos gramáticas principales: la gramática base de Jison (Meta-Lenguaje) y la gramática de prueba LL(1) de Expresiones Aritméticas.

4.1. Meta-Gramática Jison (Para reconocer el lenguaje Wison)

Esta gramática interna (no visible para el usuario final) es la encargada de leer el archivo de configuración .wison.

- **Lógica:** Reconoce palabras reservadas como Lex, Syntax, Terminal, No_Terminal y separadores de bloque.
- **Acciones Semánticas:** Al leer las producciones del usuario, extrae la fila y la columna (`this._$.first_line`) y empuja los datos a los arreglos que posteriormente consumirá la clase AnalizadorLL. Esta gramática permite comentarios de bloque `/** */` y de línea `#` ignorándolos dinámicamente en su analizador léxico interno.

4.2. Gramática de Prueba: Expresiones Aritméticas (LL1)

Esta es la gramática construida usando el lenguaje Wison para probar el motor. Reconoce sumas, restas, multiplicaciones, divisiones y agrupaciones por paréntesis, respetando la precedencia de operadores.

Código de la Gramática:

```
%_Expresion <= %_Termino %_ExpresionPrima ;
```

```
%_ExpresionPrima <= $_Mas %_Termino %_ExpresionPrima
```

```
    | $_Menos %_Termino %_ExpresionPrima
```

```
    | ;
```

```

%_Termino <= %_Factor %_TerminoPrima ;

%_TerminoPrima <= $_Mult %_Factor %_TerminoPrima
    | $_Div %_Factor %_TerminoPrima
    | ;

%_Factor <= $_Abre %_Expresion $_Cierra
    | $_ID
    | $_NUM ;

```

Explicación de su diseño:

Para que la precedencia matemática funcione de abajo hacia arriba, se dividió la estructura jerárquicamente en **Expresión** (sumas y restas), **Término** (multiplicaciones y divisiones) y **Factor** (números, variables y paréntesis).

Originalmente, una gramática aritmética natural se vería así: `%_Expresion <= %_Expresion $_Mas %_Termino`. Sin embargo, **esto tiene recursividad por la izquierda**, lo que colapsaría nuestro motor LL(1) (provocando un desbordamiento de pila o *Maximum call stack size exceeded*).

Para solucionarlo, se aplicó el procedimiento de refactorización descrito en el paso 3. Se crearon los no terminales de transición **%_ExpresionPrima** y **%_TerminoPrima**, los cuales procesan el operador y el siguiente término, o en su defecto, derivan en *Epsilon* (`| ;`) para terminar la recursión de forma segura y permitir la correcta generación del árbol de derivación (AST).